



UNIVERSIDADE FEDERAL DO MARANHÃO
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIAS
ENGENHARIA DA COMPUTAÇÃO

Emanuel Lopes Silva
José Nunes de Sousa Neto
Josuel Pinheiro Barros Júnior
Louise Reis Mendes
Raianny Cristina Ferreira da Silva

**Planejamento - Sistema de Otimização e
Monitoramento de Recursos Hídricos via IoT**

São Luís
2026

Emanuel Lopes Silva - 20250071189
José Nunes de Sousa Neto - 20250071231
Josuel Pinheiro Barros Júnior - 20250071240
Louise Reis Mendes - 20250013588
Raianny Cristina Ferreira da Silva - 20250013621

Planejamento - Sistema de Otimização e Monitoramento de Recursos Hídricos via IoT

Documento apresentado como requisito parcial de avaliação à disciplina de Sistemas Distribuídos - Turma 01, no curso de Engenharia da Computação da Universidade Federal do Maranhão.

Orientador: Prof. MSc. Luiz Henrique Neves Rodrigues

São Luís
2026

Sumário

1	INTRODUÇÃO	3
1.1	Fase 2: Levantamento Bibliográfico (Semana 2)	4
1.1.1	Fundamentos de Sistemas Distribuídos	4
1.1.2	Sistemas IoT para Gestão de Reservatórios	4
1.1.3	Análise Comparativa de Protocolos: MQTT vs. HTTP	4
1.1.4	Orquestração e Conformidade Legal	5
1.1.5	Fase 3: Planejamento e Design de Arquitetura (Semanas 3-4)	5
2	METODOLOGIA DE PLANEJAMENTO	6
2.1	Mitigação de Riscos e Contingência	6
2.2	Estratégia de Fail-over em Notificações	6
2.3	Conformidade Legal e Infraestrutura	6
3	STATUS ATUAL E PROVA DE CONCEITO (POC)	8
3.1	Validação de Hardware e Filtragem Local	8
3.2	Comunicação Distribuída e JSON	9
3.3	Transição para o Middleware Interscity	10
4	CRONOGRAMA DE EXECUÇÃO (DIAGRAMA DE GANTT)	11
4.1	Análise do Progresso Atual	11
4.2	Visualização do Cronograma	11
4.3	Marcos Futuros (Milestones)	12
	REFERÊNCIAS	13

A gestão eficiente de recursos hídricos em ambientes urbanos tornou-se um dos pilares fundamentais para o desenvolvimento de cidades inteligentes. Em centros urbanos como São Luís, a monitoração de reservatórios enfrenta desafios críticos, principalmente no que tange à falta de dados precisos e à alta latência na comunicação de falhas operacionais. O gerenciamento inadequado desses recursos não apenas compromete o abastecimento, mas resulta em desperdício de água e custos operacionais elevados para a administração pública.

Este projeto propõe o desenvolvimento de um sistema de hardware e software baseado em Internet das Coisas (IoT) para o monitoramento hídrico em tempo real. A solução busca preencher a lacuna existente entre a coleta de dados de campo e a tomada de decisão gerencial, fornecendo visualização imediata e alertas automatizados via canais de comunicação populares, como o WhatsApp.

A motivação acadêmica reside na aplicação prática de conceitos fundamentais de sistemas distribuídos, processamento de sinais e arquitetura de redes em um cenário de infraestrutura urbana real. O sistema é concebido sob uma arquitetura distribuída escalável, dividida em quatro camadas principais que garantem a robustez e a integridade dos dados:

1. **Camada de Aquisição:** Responsável pela coleta de dados físicos através de sensores ultrassônicos e de pressão integrados ao microcontrolador ESP32.
2. **Camada de Processamento Local:** Implementa firmware em linguagem C para a filtragem de ruídos (debounce) e conversão métrica dos dados brutos, garantindo a portabilidade do código para diferentes arquiteturas.
3. **Camada de Integração:** Atua como um middleware municipal, recebendo dados formatados em JSON via protocolos HTTP ou MQTT para armazenamento e processamento centralizado.

4. **Camada de Notificação:** Utiliza a orquestração via Docker para hospedar a Evolution API, permitindo o disparo automatizado de alertas críticos de forma isolada e segura.

Dessa forma, o projeto não visa apenas a automação técnica, mas a criação de um ecossistema distribuído que promova a transparência e a eficiência na gestão hídrica, mitigando riscos de desabastecimento através de notificações em tempo real e análise estatística dos dados coletados.

1.1 Fase 2: Levantamento Bibliográfico (Semana 2)

A equipe realizou uma pesquisa aprofundada sobre o estado da arte em sistemas distribuídos aplicados ao monitoramento de recursos hídricos. O levantamento focou na fundamentação teórica de sistemas distribuídos, na integração de dispositivos de borda e na conformidade com as regulamentações de dados vigentes.

1.1.1 Fundamentos de Sistemas Distribuídos

O projeto baseia-se nos conceitos clássicos de transparência, escalabilidade e comunicação em sistemas distribuídos, conforme discutido por (COULOURIS et al., 2013) e (TANENBAUM; STEEN, 2007). Estas bases teóricas são essenciais para garantir que a troca de mensagens entre os dispositivos de borda (ESP32) e o middleware municipal ocorra de forma confiável e eficiente.

1.1.2 Sistemas IoT para Gestão de Reservatórios

Trabalhos como o de (SANTOS; SILVA, 2022) descrevem a implementação de redes de sensores sem fio para medição de nível em áreas urbanas. O uso do microcontrolador ESP32 é central na arquitetura, e suas capacidades de programação de baixo nível para IoT são exploradas com base em (MAZIDI et al., 2022), permitindo o processamento paralelo para gerenciar a amostragem de dados e a pilha de protocolos de rede simultaneamente.

1.1.3 Análise Comparativa de Protocolos: MQTT vs. HTTP

A literatura técnica, exemplificada por (OLIVEIRA, 2021), aponta que para sistemas distribuídos de larga escala, o protocolo MQTT apresenta vantagens significativas em consumo de banda. No entanto, para a integração com middlewares RESTful, o uso de JSON sobre HTTP foi selecionado pela facilidade de implementação e compatibilidade com APIs de mensageria modernas.

1.1.4 Orquestração e Conformidade Legal

A utilização de Docker para isolar serviços de mensageria, como a Evolution API, permite que o sistema de notificação seja independente da infraestrutura de hardware (LIMA, 2023). Adicionalmente, toda a lógica de disparo de alertas via WhatsApp é projetada para estar em total conformidade com a Lei Geral de Proteção de Dados (LGPD) (BRASIL, 2018), assegurando a privacidade e a segurança no tratamento das informações dos usuários e gestores.

1.1.5 Fase 3: Planejamento e Design de Arquitetura (Semanas 3-4)

O planejamento da arquitetura foi estruturado sob uma perspectiva de sistemas distribuídos, priorizando a disponibilidade e a integridade dos dados hídricos. A topologia do sistema foi desenhada para operar com redundância na camada de aquisição, utilizando simultaneamente o sensor ultrassônico HC-SR04 e um sensor de pressão hidrostática. Esta configuração permite que o sistema mantenha a operabilidade caso um dos componentes sofra falha ou desgaste acelerado em ambiente úmido.

A modularização do firmware é um dos pilares deste planejamento. O desenvolvimento foi segmentado em blocos lógicos, com destaque para o módulo `analise_dados.c`, que implementa algoritmos de filtragem digital, como a média móvel. Este tratamento estatístico local é fundamental para eliminar ruídos de leitura causados pela turbulência da água ou interferências de sinal, garantindo que apenas dados validados sejam serializados em formato JSON e transmitidos via protocolos HTTP ou MQTT para o middleware municipal.

Para fins de acompanhamento técnico e auditoria do progresso do firmware e da orquestração Docker, todo o código-fonte e histórico de commits estão disponíveis no repositório oficial do projeto no GitHub (SILVA et al., 2026).

Tabela 1 – Dados de Identificação do Projeto

Item	Descrição
Nome do Projeto	Sistema de Otimização e Monitoramento de Recursos Hídricos via IoT
Repositório GitHub	Link do Repositório no GitHub

Metodologia de Planejamento

A metodologia adotada para o planejamento deste projeto baseia-se na antecipação de falhas e na conformidade com restrições técnicas e legais. O fluxo de trabalho foi estruturado para mitigar os riscos inerentes à operação de hardware em ambientes críticos e à instabilidade de redes de comunicação distribuídas.

2.1 Mitigação de Riscos e Contingência

Para enfrentar o risco de oxidação de hardware, definiu-se a utilização de sensores com grau de proteção IP67. Quanto à instabilidade de rede (Wi-Fi/GPRS), o planejamento prevê a implementação de um buffer local no firmware em C, permitindo que os pacotes JSON sejam armazenados temporariamente e reenviados assim que a conectividade for restabelecida, evitando a perda de dados históricos.

2.2 Estratégia de Fail-over em Notificações

Um diferencial crítico da metodologia é a estratégia de “Fail-over” para o sistema de alertas. Em caso de indisponibilidade ou bloqueio da Evolution API (WhatsApp), o sistema está projetado para comutar automaticamente para canais secundários, como o Telegram ou um frontend web dedicado. Esta redundância na camada de notificação assegura que as informações sobre níveis críticos de reservatórios cheguem aos gestores sem interrupções.

2.3 Conformidade Legal e Infraestrutura

Toda a infraestrutura de software, incluindo o Middleware e a Evolution API, é orquestrada via Docker Compose, garantindo um ambiente de execução isolado e escalável. Além disso, o tratamento de dados (como números de telefone de gestores) segue

rigorosamente os preceitos da Lei Geral de Proteção de Dados (LGPD), com restrições técnicas aplicadas ao armazenamento de logs e informações sensíveis no bot.

Status Atual e Prova de Conceito (PoC)

No estágio atual do projeto, a equipe concluiu com sucesso a Prova de Conceito técnica para validar os componentes críticos da Camada de Aquisição e Processamento Local. É importante ressaltar que a montagem em bancada e a simulação via Wokwi foram utilizadas estritamente para validação inicial de hardware e algoritmos.

Utilizando um microcontrolador ESP32 e o sensor ultrassônico HC-SR04, foi validada a lógica de leitura hídrica. Implementou-se um Filtro de Média Móvel com 10 amostras no firmware para mitigar ruídos de leitura causados por turbulência ou interferências ambientais, garantindo a precisão dos dados antes da transmissão.

Para validar a pilha de comunicação (HTTP/JSON), os dados foram inicialmente enviados para o serviço *Webhook.site*, servindo como um “mock” de recepção para confirmar a integridade dos pacotes. Com a lógica de rede devidamente homologada, o projeto entra agora na fase de integração final, onde o fluxo de dados será migrado do ambiente de teste para o Middleware Interscity. Esta plataforma será o nó central do sistema distribuído, responsável pela persistência escalável e pela disponibilização dos dados para a camada de notificação.

3.1 Validação de Hardware e Filtragem Local

Foi realizada a montagem do protótipo com ESP32 e sensor HC-SR04. O firmware integra um Filtro de Média Móvel (10 amostras), conforme planejado para o módulo `analise_dados.c`.

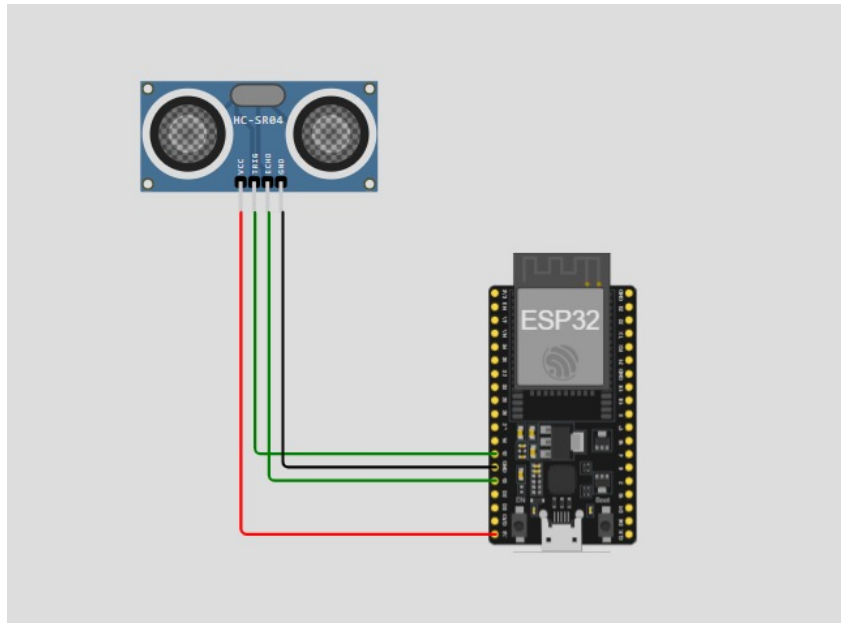


Figura 1 – Protótipo de bancada para validação da Camada de Aquisição.

3.2 Comunicação Distribuída e JSON

Os dados foram formatados em JSON e transmitidos via HTTP POST. A validação do fluxo de rede foi feita utilizando um middleware de teste (*Webhook.site*), garantindo a recepção correta dos pacotes.

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Conectando ao WiFi.....
[SISTEMA HÍDRICO UFMA] Conectado com sucesso!
Nível Médio Calculado: 145.20 cm
Status do Envio: 200
Nível Médio Calculado: 145.18 cm
Status do Envio: 200
Nível Médio Calculado: 145.18 cm
Status do Envio: 200
Nível Médio Calculado: 145.18 cm
Status do Envio: 200
Nível Médio Calculado: 145.19 cm
Status do Envio: 200
Nível Médio Calculado: 145.19 cm
Status do Envio: 200
Nível Médio Calculado: 145.18 cm
Status do Envio: 200
```

Figura 2 – Log do Serial Monitor confirmando o cálculo da média e status de envio 200.

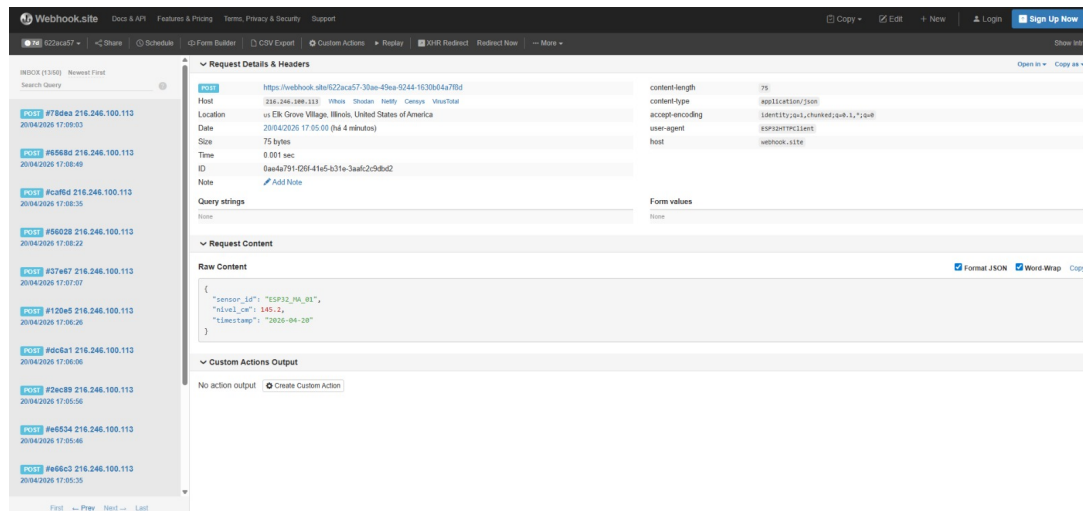


Figura 3 – Visualização do payload JSON recebido no endpoint de teste.

3.3 Transição para o Middleware Interscity

Embora os testes iniciais de conectividade tenham utilizado o serviço *Webhook.site* para monitoramento de pacotes via HTTP (Status 200), o restante do desenvolvimento será integrado ao **Middleware Interscity**.

Esta transição marca o fim da fase de testes isolados e o início da operação distribuída, onde o Interscity atuará como a camada de integração municipal, orquestrando o recebimento dos dados para posterior processamento e disparo de alertas via Evolution API.

Cronograma de Execução (Diagrama de Gantt)

O cronograma do projeto foi estruturado para cobrir um ciclo de 14 semanas, garantindo a integração contínua entre as camadas de hardware e software propostas. O planejamento utiliza o Gráfico de Gantt como ferramenta principal para monitorar o caminho crítico e a simultaneidade das tarefas.

4.1 Análise do Progresso Atual

Até a presente data (20 de abril de 2026), o projeto apresenta um progresso satisfatório, com as fases iniciais de Concepção, Levantamento Bibliográfico e Design de Arquitetura concluídas com 100% de aproveitamento.

Atualmente, a equipe encontra-se na fase de **Execução**, iniciada em 20/04, com foco no desenvolvimento do Firmware do ESP32 e na configuração da infraestrutura Docker. Ambas as atividades já registram 10% de conclusão, refletindo o início efetivo da codificação e orquestração de serviços.

4.2 Visualização do Cronograma

O diagrama abaixo detalha a sucessão das etapas de integração da Evolution API, testes de estresse e o encerramento previsto para o final de junho.

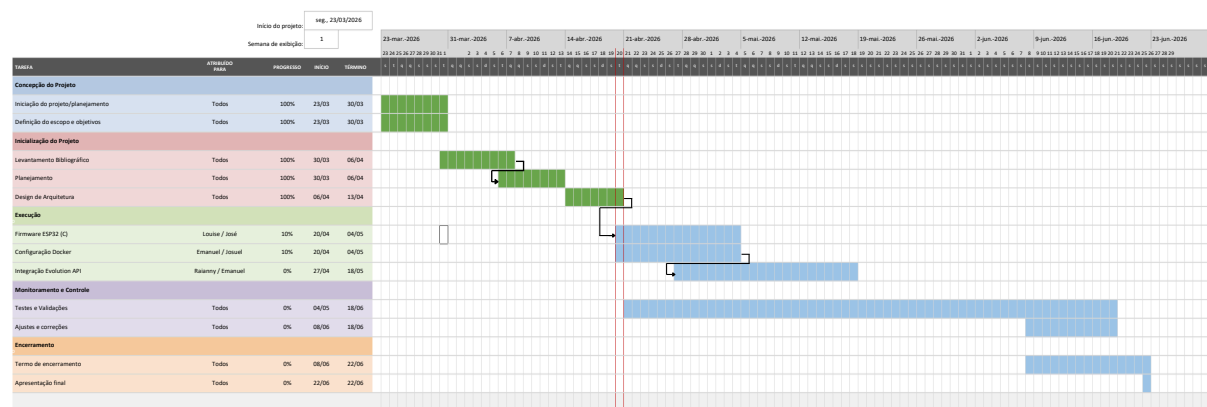


Figura 4 – Diagrama de Gantt atualizado em 20/04/2026 detalhando as fases de execução e integração.

4.3 Marcos Futuros (Milestones)

- **27/04 a 18/05:** Início da integração com a Evolution API para notificações via WhatsApp.
- **04/05 a 18/06:** Fase extensa de testes e validações do projeto.
- **22/06:** Encerramento formal e Apresentação Final do protótipo funcional.

Referências

- BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União. Brasília, DF.
- COULOURIS, G. et al. *Sistemas Distribuídos: Conceitos e Projeto*. 5. ed. Porto Alegre: Bookman, 2013.
- LIMA, V. C. Arquiteturas de microserviços e Docker para notificações em tempo real em sistemas IoT. *Revista de Sistemas Distribuídos e Redes*, v. 12, n. 2, p. 45–60, 2023.
- MAZIDI, M. A. et al. *The ESP32 Microcontroller: Embedded Programming for IoT*. 1. ed. [S.l.]: MicroDigitalEd, 2022.
- OLIVEIRA, A. F. *Avaliação de Performance entre MQTT e HTTP em Dispositivos de Borda para Smart Cities*. Dissertação (Dissertação (Mestrado em Engenharia da Computação)) — Universidade Federal de Santa Catarina, 2021.
- SANTOS, J. P.; SILVA, R. M. Monitoramento de reservatórios urbanos utilizando ESP32 e redes de sensores. In: *Anais do Simpósio Brasileiro de Sistemas Computacionais (SBSC)*. [S.l.: s.n.], 2022.
- SILVA, E. L. et al. *Sistema de Otimização e Monitoramento de Recursos Hídricos via IoT*. [S.l.]: GitHub, 2026. <https://github.com/emanuelsilva69/sist.distribuidos-medidor_de_gua_caixa-bueiro>. Acedido em: 20 abr. 2026.
- TANENBAUM, A. S.; STEEN, M. V. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. Rio de Janeiro: LTC, 2007.